CODEVOYAGE

A Gamified Java 21 Learning Platform with Sandboxed Execution & Adaptive Analytics

COMPREHENSIVE PROJECT REPORT & TECHNICAL ARCHITECTURE SPECIFICATION
Submitted in partial fulfillment of the requirements for the Degree of
Bachelor of Technology / Bachelor of Engineering in Computer Science and Engineering

Candidate Name / Author:	[Editable Placeholder: Cadet / Student Name] (Roll No: [Editable Placeholder])
Project Supervisor / Guide:	[Editable Placeholder: Faculty Supervisor / Professor Name, Ph.D.]
Department / Academic Unit:	Department of Computer Science and Engineering
Institution / University:	[Editable Placeholder: Institute / University Name, City, Country]
Technology Stack & Version:	Java JDK 21 LTS | Spring Boot 3.2.5 | React 19 | Node.js Sandbox | PostgreSQL 16

Academic Year 2026 — 2027 | Document Version 1.0.0 (Production Master)

Document Control

This document establishes the official technical, pedagogical, and architectural specification for CodeVoyage. The metadata and version history recorded below govern the deployment, auditing, and maintenance of the software artifact.

Revision	Release Date	Author / Role	Primary Changes & Scope
v0.1.0	March 2026	Engineering Team	Initial prototype, Monaco editor integration, and single-world mission runner.
v0.5.0	June 2026	System Architect	Microservice sandbox isolation, Spring Security JWT authentication filter.
v0.9.0	September 2026	Full-Stack Team	Learning analytics dashboard, radar mastery calculations, and diagnostic hints.
v1.0.0	October 2026	Lead Architect	Final verification: 4 worlds, 20 canonical missions, idempotent replay, Pre/Post test CSV exporter, and 30 passing integration test cases.

Document Verification & Sign-off

Review Role	Reviewer Name & Designation	Verification Verdict	Signature / Date
Project Lead	[Editable Placeholder: Lead Student Engineer]	Verified & Validated	_____
Technical Guide	[Editable Placeholder: Project Guide / Assistant Professor]	Approved for Examination	_____
External Examiner	[Editable Placeholder: External Academic Evaluator]	Accepted	_____

Abstract

Introductory programming courses in higher education face severe attrition rates due to cognitive overload, cryptic compiler feedback, and disengaging pedagogical tools. Beginners learning Java are routinely overwhelmed by ceremonial syntax—such as public static void main, strict type systems, and import dependencies—before experiencing the creative empowerment of computational problem-solving. This project presents CodeVoyage, an immersive, game-based educational platform that teaches authentic Java (JDK 21) within a space-exploration narrative.

CodeVoyage departs from conventional passive tutorials and simplistic block-based simulations by hosting a production-grade coding environment powered by Microsoft's Monaco Editor and an isolated multi-process execution sandbox. The system architecture segregates the presentation tier (React 19 with Vite, TypeScript, and Tailwind CSS), the enterprise application tier (Spring Boot 3.2.5 with Spring Security and Hibernate), and an unprivileged Node.js/OpenJDK execution service that enforces rigid memory caps (128 MB) and execution timeouts (5000 ms) to neutralize malicious code and infinite loops.

The learning progression spans four distinct celestial sectors containing 20 canonical missions: World 1 (Ignis Station: Syntax, Variables, Scanner I/O, Arithmetic Operators, Conditionals), World 2 (Kryon Belt: Relational Logic, Logical Operators, Switch Decisions, Nested Control), World 3 (Nova Sector: While Loops, For Loops, Do-While Loops, Control Flow Break/Continue, Matrix Boss), and World 4 (Function Nebula: Static Methods, Return Values, Array Allocation, Linear Search & Peak Finding, Quantum Nebula Boss). Each mission guides the student through an eight-stage pedagogical loop: contextual narrative story, illustrated concept lesson, interactive diagnostic practice questions, hands-on Java challenge, sandboxed compilation, automated unit test evaluation, diagnostic error translation, and sequential progression.

An advanced Learning Analytics Engine dynamically monitors student telemetry, calculating multidimensional concept mastery, compiler pass rates, practice accuracy, hint reliance, and study duration. Furthermore, an integrated Academic Evaluation Subsystem computes pre-test and post-test normalized learning gain (Hake's g) and exports anonymized empirical datasets for educational research. Comprehensive verification comprising 30 backend JUnit/Surefire integration tests, strict TypeScript compilation, and live REST API validation

confirms that CodeVoyage delivers persistent, account-isolated, and replay-safe programming instruction.

Keywords: Gamified Learning, Computer Science Education, Java 21, Sandboxed Execution, Spring Boot, React, Learning Analytics, Educational Technology, Normalized Learning Gain.

Table of Contents

The comprehensive table of contents outlines all preliminary pages, 19 core technical chapters, research references, and appendices. In Microsoft Word, press F9 on any field to refresh dynamic pagination.

Chapter / Section Title	Page
Preliminary Pages	i
Document Control	ii
Abstract	iii
Table of Contents	iv
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
Chapter 1 — Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Why Conventional Programming Interfaces Fail Beginners	2
1.4 Proposed Solution: CodeVoyage	3
1.5 Project Objectives	3
1.6 Scope of the Project	4
1.7 Intended Users & Expected Benefits	4
Chapter 2 — Existing System and Proposed System	5
2.1 Conventional Approaches to Teaching Java	5
2.2 Limitations of Existing Solutions	5
2.3 The Proposed CodeVoyage Paradigm	6
2.4 Structural Comparison	6
2.5 Architectural Trade-offs	7
Chapter 3 — Requirements Analysis	8
3.1 Functional Requirements	8
3.2 Non-Functional Requirements	9
3.3 User Roles and Permissions	10
3.4 Requirements Traceability Matrix	10
Chapter 4 — Technology Stack	12

Chapter / Section Title	Page
4.1 Comprehensive Technology Stack Specification	12
Chapter 5 — System Architecture	14
5.1 High-Level Architecture	14
5.2 Three-Tier Component Communication Architecture	15
5.3 Authentication & JWT Security Flow	16
5.4 Error Handling & Fault Isolation	17
Chapter 6 — Frontend Design and User Interface	18
6.1 Cadet User Journey Architecture	18
6.2 Navigation Structure and Screen Catalog	19
6.3 Component Organization & Reusable UI Modules	20
6.4 State Management & Context Architecture	21
6.5 Visual Design System	21
Chapter 7 — Educational Methodology and Java Curriculum	22
7.1 Pedagogical Framework & Learning Sequence	22
7.2 Complete 20-Mission Curriculum Specification	23
7.2.1 World 1 — Ignis Station (Foundations, I/O & Branching)	23
7.2.2 World 2 — Kryon Belt (Relational Logic & Switch Decisions)	24
7.2.3 World 3 — Nova Sector (Loops, Iteration & Control Flow)	25
7.2.4 World 4 — Function Nebula (Methods, Modular Design & Arrays)	26
Chapter 8 — Game World and Gamification Mechanics	27
8.1 Galaxy Map & Sector Progression Architecture	27
8.2 World Unlock Prerequisite Rules	28
8.3 Gamification Engine: XP, Ranks & Non-Punishing Stars	28
8.4 Starfleet Commendations & Achievements	29
8.5 Replay Safety & Persistence Across Sessions	30
Chapter 9 — Java Coding and Execution Engine	31
9.1 Monaco Code Editor Integration	31
9.2 Sandboxed Execution Pipeline	31
9.3 Resource Limits & Infinite Loop Protection	32
9.4 Test Case Evaluation: Visible vs Hidden Tests	33
9.5 Diagnostic Error Translation Layer	33
Chapter 10 — Database Design	34

Chapter / Section Title	Page
10.1 Entity-Relationship Architecture	34
10.2 Primary Data Dictionaries	35
Chapter 11 — Backend Architecture and APIs	37
11.1 Layered Architecture Pattern	37
11.2 Complete REST API Endpoint Catalogue	37
Chapter 12 — Authentication and Security	39
12.1 Authentication & Password Hashing	39
12.2 Stateless JWT Token Architecture	39
12.3 Code Execution Sandboxing & Security Boundaries	39
12.4 Security Risk Analysis & Hardening Recommendations	40
Chapter 13 — Learning Analytics and Evaluation	41
13.1 Learning Analytics Architecture	41
13.2 Core Learning Metrics & Formulas	41
13.3 Diagnostic Error Categorizer	42
13.4 Academic Pre-Test, Post-Test & Normalized Gain Subsystem	43
Chapter 14 — Major Workflows and Sequence Diagrams	44
14.1 Code Submission & Real-Time Feedback Workflow	44
14.2 Detailed Step-by-Step System Workflows	45
Chapter 15 — Testing and Verification	46
15.1 Testing Strategy & Methodology	46
15.2 Automated Backend Test Results (30 Test Cases)	46
15.3 Frontend Static Type Checking Verification	48
Chapter 16 — Installation and Execution Guide	49
16.1 System Prerequisites	49
16.2 Native Three-Terminal Execution Guide	49
16.3 Production Multi-Container Docker Launch	50
16.4 Troubleshooting Guide	51
Chapter 17 — Implementation Status and Known Issues	52
17.1 Implementation Status Matrix	52
Chapter 18 — Limitations and Future Scope	54
18.1 Architectural & Curriculum Limitations	54
18.2 Proposed Future Enhancements	54

Chapter / Section Title	Page
Chapter 19 — Conclusion	56
19.1 Summary of Accomplishments	56
19.2 Concluding Remarks	57
References	58
Appendix A — API Reference Summary	59
Appendix B — Database Dictionary	60
Appendix C — Environment Variables Reference	61
Appendix D — Technical Glossary	62

List of Figures

Figure 5.1: High-Level System Architecture and Multi-Tier Service Topology

Figure 5.2: Three-Tier Component Communication Architecture and Protocols

Figure 5.3: Authentication & Stateless JWT Security Flow

Figure 6.1: Cadet User Journey Architecture and Screen State Machine

Figure 7.1: Pedagogical Learning Loop Sequence (Mission to Flight Clearance)

Figure 8.1: Mission Progression & World Unlocking State Machine

Figure 9.1: Sandboxed Java Execution, Process Isolation, and Grading Pipeline

Figure 10.1: Entity-Relationship Diagram (Core JPA Domain Model)

Figure 13.1: Learning Analytics and Intelligence Architecture

Figure 14.1: Code Submission & Real-Time Feedback Sequence Diagram

List of Tables

Table 3.1: Functional Requirements Specification and Verification Matrix

Table 3.2: Non-Functional System Requirements and Quality Attributes

Table 3.3: Requirements Traceability Matrix Linking Specifications to Code Evidence

Table 4.1: Comprehensive Technology Stack, Frameworks, and Architectural Justifications

Table 7.1: Complete Java Curriculum Specification: World 1 — Ignis Station (M01–M05)

Table 7.2: Complete Java Curriculum Specification: World 2 — Kryon Belt (M06–M10)

Table 7.3: Complete Java Curriculum Specification: World 3 — Nova Sector (M11–M15)

Table 7.4: Complete Java Curriculum Specification: World 4 — Function Nebula (M16–M20)

Table 8.1: Gamification Ranks, Level Thresholds, and Cumulative XP Progression

Table 8.2: Starfleet Commendations and Achievement Directive Catalog

Table 10.1: Primary Data Dictionary: users Table Schema

Table 10.2: Primary Data Dictionary: user_progress Table Schema

Table 10.3: Primary Data Dictionary: missions Table Schema

Table 10.4: Primary Data Dictionary: submissions Table Schema

Table 11.1: REST API Endpoint Catalog and Security Permissions

Table 13.1: Learning Analytics Metrics, Operational Formulas, and Interpretations

Table 15.1: Automated Backend Test Suite Execution Matrix (30 Test Cases)

Table 16.1: Troubleshooting Guide for Local Setup, Sandbox, and Database Errors

Table 17.1: Comprehensive Implementation Status, Verification, and Defect Resolution Matrix

List of Abbreviations

Abbreviation	Full Expanded Term
API	Application Programming Interface
AST	Abstract Syntax Tree
BCrypt	Blowfish-based Password Hashing Algorithm
CLI	Command Line Interface
CPU	Central Processing Unit
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DTO	Data Transfer Object
ERD	Entity-Relationship Diagram
FK	Foreign Key
H2	Embedded Java Relational Database Management System
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
HUD	Heads-Up Display
IDE	Integrated Development Environment
JDK	Java Development Kit
JPA	Java Persistence API
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
JWT	JSON Web Token
LTS	Long-Term Support
MCQ	Multiple Choice Question
ORM	Object-Relational Mapping
OS	Operating System
PK	Primary Key
RAM	Random Access Memory
REST	Representational State Transfer
SDK	Software Development Kit
SHA	Secure Hash Algorithm

Abbreviation	Full Expanded Term
SPA	Single Page Application
SQL	Structured Query Language
TOC	Table of Contents
UI	User Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UX	User Experience
VM	Virtual Machine
XP	Experience Points

Chapter 1 — Introduction

1.1 Background and Motivation

Computer science and software engineering education have become foundational disciplines in global higher education. Among modern enterprise programming languages, Java remains a dominant industry pillar, powering enterprise backend services, distributed systems, financial trading engines, and Android mobile applications. However, introductory programming education frequently represents an insurmountable hurdle for novice learners. Universities worldwide report high student attrition rates in entry-level computer science coursework, with difficulty in mastering programming syntax and computational abstractions cited as primary factors.

Traditional introductory programming pedagogy relies heavily on static textbooks, passive slide lectures, and command-line execution interfaces. When beginners attempt to run their first Java program, they are confronted with complex boilerplate code—including public static void main(String[] args), class declarations, package structures, and compiler error diagnostics. Novice students often spend hours diagnosing missing semicolons, case-sensitivity discrepancies, or environment configuration problems rather than developing computational thinking skills.

1.2 Problem Statement

The fundamental problem addressed by this project can be stated as follows: Conventional programming instruction lacks contextual engagement, provides cryptic compiler diagnostics, and requires complex local toolchain installations that intimidate beginners. Furthermore, existing online coding platforms either oversimplify programming using drag-and-drop visual blocks (failing to teach authentic industry syntax) or present stark, gameless competitive coding interfaces that induce anxiety and fatigue. Consequently, there is an urgent need for an educational platform that combines authentic, text-based Java (JDK 21) programming with immersive narrative gamification, safe browser-based compilation, contextual AI companionship, and granular learning analytics.

1.3 Why Conventional Programming Interfaces Fail Beginners

Extensive computing education research identifies four major failure modes in conventional beginner programming environments:

- **1.** Cognitive Overload from Ceremonial Boilerplate: Beginners must memorize rigid structural rules before they understand what variables, expressions, or loops accomplish.
- **2.** Unhelpful Compiler Stack Traces: Standard Java compiler messages, such as unclosed string literal or non-static variable cannot be referenced from a static context, are designed for veteran developers rather than novices.
- **3.** Lack of Intrinsic Narrative Motivation: Exercises such as 'compute the factorial of N' or 'print Fibonacci numbers' lack contextual relevance, reducing learning to dry mathematical chores.
- **4.** Security Vulnerabilities in Remote Execution: Safely executing arbitrary student code on university servers requires complex sandboxing that ordinary educational websites omit.

1.4 Proposed Solution: CodeVoyage

CodeVoyage resolves these challenges by situating Java programming within an interactive space-exploration adventure. Students assume the role of Starfleet Cadets navigating celestial sectors. Every mission presents an authentic shipboard crisis—such as reactor coolant failure, thruster miscalibration, or asteroid field scanning—that can only be resolved by writing, compiling, and executing real Java code. The platform features an embedded Monaco Code Editor, an isolated multi-process execution sandbox, an encouraging holographic robot companion named B.Y.T.E., non-punishing star evaluations, and an intelligence dashboard tracking mastery across the Java curriculum.

1.5 Project Objectives

The engineering and educational objectives of CodeVoyage are:

- **Objective 1:** To engineer an immersive, responsive single-page web application using React 19, TypeScript, and Tailwind CSS featuring a futuristic space HUD aesthetic.
- **Objective 2:** To implement an enterprise-grade backend with Spring Boot 3.2.5, providing secure RESTful APIs, JWT stateless authentication, and relational persistence.
- **Objective 3:** To construct an isolated, secure Java (JDK 21) compilation sandbox that enforces rigid memory caps (128 MB) and execution timeouts (5000 ms).
- **Objective 4:** To design and deploy a sequential 20-mission Java curriculum across four celestial worlds, encompassing syntax, primitives, I/O, control flow, loops, methods, and arrays.
- **Objective 5:** To provide a diagnostic error translation layer that translates cryptic javac and JVM stack traces into clear, actionable advice.
- **Objective 6:** To develop a Learning Analytics and Academic Evaluation Subsystem that computes multidimensional mastery, identifies struggling concepts, and exports empirical research datasets.

1.6 Scope of the Project

The scope of CodeVoyage encompasses the full software development lifecycle of the web application, including frontend interfaces, backend services, database schema design, sandbox security boundaries, automated test suites, and empirical evaluation metrics. The platform covers foundational imperative and procedural Java concepts. Advanced object-oriented topics (inheritance, interfaces, polymorphism, multithreading, and stream APIs) are recognized as future curriculum expansions.

1.7 Intended Users & Expected Benefits

The primary users are undergraduate computer science students, coding bootcamp attendees, and self-directed programming novices. Secondary users include university professors, teaching assistants, and institutional researchers who utilize the analytics and evaluation datasets. Expected benefits include accelerated concept acquisition, reduced syntax frustration, higher course completion rates, and an engaging, stress-free learning atmosphere.

Chapter 2 — Existing System and Proposed System

2.1 Conventional Approaches to Teaching Java

Traditional Java instruction relies on desktop integrated development environments (e.g., Eclipse, IntelliJ IDEA, NetBeans) or simple command-line terminal compilation (javac and java). Students read theoretical textbook chapters, observe instructor demonstrations, and attempt weekly programming assignments. Alternatively, web-based platforms fall into two categories: block-based programming environments (such as Scratch or Blockly) or competitive coding portals (such as LeetCode, HackerRank, or Codeforces).

2.2 Limitations of Existing Solutions

A systematic analysis reveals critical shortcomings across current pedagogical solutions:

- **1. Visual Disconnect:** Block-Based Platforms: While Scratch and Blockly reduce syntax anxiety, they do not teach real Java text syntax. Students experience a severe disconnect when transitioning from visual blocks to text-based coding.
- **2. Punitive Interfaces:** Competitive Coding Portals: LeetCode and HackerRank are designed for algorithmic interview preparation rather than novice instruction. They lack structured lessons, offer zero contextual narrative, and penalize failed attempts, creating anxiety.
- **3. Lack of Structure:** Generic Online Compilers: In-browser compilers (e.g., JDoodle, OnlineGDB) provide basic execution but lack structured curricula, automated test case grading, gamification mechanics, and learning analytics.
- **4. Setup Overhead:** Local Environment Friction: Installing JDKs, configuring JAVA_HOME environment variables, and navigating complex IDE interfaces consumes vital classroom hours and causes early frustration.

2.3 The Proposed CodeVoyage Paradigm

CodeVoyage bridges the gap between playful engagement and professional software development. By integrating Microsoft's Monaco Editor directly into a sci-fi cockpit HUD, students write authentic Java 21 code with syntax highlighting, autocomplete, and line numbers. The game mechanics provide immediate intrinsic rewards—reparing starship thrusters, stabilizing plasma cores, and unlocking new planetary sectors—without punishing learners for syntax mistakes.

2.4 Structural Comparison

Evaluation Dimension	Traditional Desktop IDE	Competitive Coding Portal	CodeVoyage Platform

Evaluation Dimension	Traditional Desktop IDE	Competitive Coding Portal	CodeVoyage Platform
Installation Overhead	Heavy (JDK, IDE, paths)	None (Browser-based)	Zero (Instant Web Browser Cockpit)
Authentic Java Syntax	Yes (Full Java)	Yes (Full Java)	Yes (JDK 21 LTS with Monaco Editor)
Narrative & Theme	None (Blank editor)	None (Algorithmic problem)	Rich Sci-Fi Space Exploration World
Companion / Hints	None (Cryptic stack traces)	Community discussion / paid	B.Y.T.E. Holographic AI Companion
Evaluation Mindset	Grade / Pass / Fail	Time / Memory percentile	Constructive 1-3 Stars + XP Progression
Analytics Engine	None	Submission history graph	Multi-concept mastery radar & diagnostics
Academic Research Export	None	None (Proprietary)	Standardized Pre/Post Normalized Gain CSV

2.5 Architectural Trade-offs

Adopting a cloud-based sandbox architecture introduces specific engineering trade-offs. While browser-based execution eliminates client installation friction, it imposes server-side execution burdens and security risks. CodeVoyage mitigates these risks through process isolation, memory capping, and timeout enforcement, trading arbitrary OS-level file access for guaranteed host integrity.

Chapter 3 — Requirements Analysis

3.1 Functional Requirements

The system's functional requirements define the core operational capabilities implemented in the software:

Req ID	Requirement Description	Subsystem	Verification Status
FR-01	Cadet Registration & Login with encrypted passwords and JWT token issuance	Auth Module	Implemented & Verified
FR-02	Galaxy Map rendering 4 celestial worlds with accurate sequential unlock states	Galaxy Map UI	Implemented & Verified
FR-03	Curriculum display of 20 canonical missions with lesson, practice, and challenge	Mission Engine	Implemented & Verified
FR-04	In-browser Java code editing using Monaco Editor with syntax diagnostics	Frontend Editor	Implemented & Verified
FR-05	Sandbox execution of Java 21 code with process isolation and timeout limits	Execution Sandbox	Implemented & Verified
FR-06	Automated grading against visible and hidden unit test cases	Submission Engine	Implemented & Verified
FR-07	Dynamic star evaluation (1-3 stars) and XP awarding based on hint usage and tries	Gamification	Implemented & Verified
FR-08	Permanent persistence of mission completions, stars, XP, and sector unlock progress	Database Layer	Implemented & Verified
FR-09	Safe mission replay allowing code practice without erasing progress or duplicating XP	Progress Service	Implemented & Verified
FR-10	Learning Analytics Dashboard computing concept mastery, pass rates, and trends	Analytics Subsystem	Implemented & Verified
FR-11	Academic evaluation engine administering Pre/Post tests and exporting CSV data	Evaluation Subsystem	Implemented & Verified
FR-12	Starfleet commendation deck awarding 13 distinct achievements upon milestones	Achievement System	Implemented & Verified

3.2 Non-Functional Requirements

Non-functional requirements specify quality attributes, performance bounds, and security constraints:

NFR ID	Attribute	Specification / Bound	Implementation Mechanism
NFR-01	Execution Latency	Sandbox compilation and run < 1500 ms for normal solutions	Pre-warmed JVM execution in Node.js child_process

NFR ID	Attribute	Specification / Bound	Implementation Mechanism
NFR-02	Resource Isolation	Memory restricted to 128 MB per execution; 5000 ms CPU timeout	JVM flag -Xmx128m and process kill timer
NFR-03	Security & Auth	Stateless HMAC-SHA256 JWT tokens; passwords hashed with BCrypt	Spring Security filter chain and BCryptPasswordEncoder
NFR-04	Data Integrity	Zero data loss across browser refreshes, server reboots, or re-logins	Relational foreign keys and transactional JPA commits
NFR-05	Tamper Resistance	XP, stars, levels, and unlocks computed strictly on backend	Server-side grading; frontend cannot forge progress
NFR-06	Usability & UX	Dark sci-fi HUD aesthetic; responsive layout across desktop screens	Tailwind CSS utility classes and Lucide vector icons

3.3 User Roles and Permissions

The system enforces role-based access control (RBAC) via the Role enum in com.codevoyage.user:

- **Cadet / Explorer:** EXPLORER (Default Student): Can authenticate, complete lessons, submit code, view personal analytics, unlock worlds, and export personal dossier records.
- **Commander / Faculty:** COMMANDER (Instructor / Faculty): Inherits all cadet permissions plus access to cohort-wide analytics (GET /api/v1/analytics/faculty) and empirical dataset export.
- **Administrator:** ADMIN (System Administrator): Unrestricted database maintenance and curriculum authoring capabilities.

3.4 Requirements Traceability Matrix

The traceability matrix maps system specifications directly to inspected source files:

Req ID	Core Requirement	Primary Implementation File	Test Evidence File
FR-01	JWT Registration / Login	backend/.../auth/AuthController.java	SubmissionServiceTest.java
FR-02	Sequential World Unlocking	backend/.../mission/MissionService.java	MissionProgressionIntegrationTest.java
FR-03	20-Mission Java Curriculum	backend/.../config/DataLoader.java	MissionProgressionIntegrationTest.java
FR-04	Monaco Editor Integration	frontend/src/editor/JavaCodeEditor.tsx	Frontend tsc --noEmit check
FR-05	Isolated Sandbox Execution	execution-service/src/index.ts	OutputNormalizerTest.java

Req ID	Core Requirement	Primary Implementation File	Test Evidence File
FR-06	Automated Test Grading	backend/.../submission/SubmissionService.java	SubmissionServiceTest.java
FR-07	Star & XP Gamification	backend/.../progress/ProgressService.java	MissionProgressionIntegrationTest.java
FR-08	Progress Persistence	backend/.../progress/UserProgress.java	MissionProgressionIntegrationTest.java
FR-09	Safe Idempotent Replay	backend/.../progress/ProgressService.java	MissionProgressionIntegrationTest.java
FR-10	Learning Analytics Engine	backend/.../analytics/AnalyticsService.java	AnalyticsServiceTest.java
FR-11	Academic Pre/Post Dataset	backend/.../evaluation/EvaluationService.java	EvaluationController live CSV test
FR-12	Achievement Commendations	backend/.../achievement/AchievementService.java	DataLoader seed verification

Chapter 4 — Technology Stack

The technology stack for CodeVoyage was chosen to ensure high responsiveness, enterprise-grade reliability, authentic compilation accuracy, and strict security isolation. Every dependency has been verified in the project repository configuration files (pom.xml, package.json, and docker-compose.yml).

Layer / Subsystem	Technology & Version	Architectural Role	Justification & Benefits
Frontend Framework	React 19.3.0	Core Single Page Application (SPA)	Component-driven reactive architecture, high performance virtual DOM, extensive UI ecosystem.
Frontend Toolchain	Vite 8.3.0	Module Bundler & Dev Server	Sub-millisecond Hot Module Replacement (HMR) and optimized Rollup production builds.
Language (Frontend)	TypeScript 6.0.2	Static Type Safety	Eliminates runtime type errors, guarantees API contract alignment, facilitates safe refactoring.
Styling System	Tailwind CSS 3.4.19	Utility-First CSS Framework	Rapid implementation of futuristic sci-fi HUD styling, responsive glassmorphism, zero runtime CSS bloat.
Code Editor Engine	Monaco Editor 4.7.0	In-Browser Java IDE	The official editor engine behind Microsoft VS Code; provides authentic Java syntax coloring, line numbers, and shortcuts.
Animation Library	Framer Motion 13.2.0	UI Transitions & Modals	Declarative physics-based animations for planet orbits, celebration modals, and cinematic launch transitions.
Icons	Lucide React 1.45.0	Vector Sci-Fi Icons	Lightweight, customizable SVG vector icons matching the starship aesthetic.
Backend Framework	Spring Boot 3.2.5	Enterprise REST API	Mature dependency injection, robust transaction management, seamless JPA data handling, production stability.
Language (Backend)	Java JDK 21 LTS	Core Platform & Target Language	Modern Long-Term Support Java release; enables modern language features (pattern matching, records, virtual threads).

Layer / Subsystem	Technology & Version	Architectural Role	Justification & Benefits
Security Framework	Spring Security 6.2.4	Authentication & RBAC	Industry-standard security architecture for endpoint interception, stateless token validation, and password hashing.
JWT Token Engine	JJWT 0.12.5	Stateless Authentication	Standards-compliant HMAC-SHA256 token creation, parsing, and cryptographic verification.
Database Layer	PostgreSQL 16 / H2	Relational Persistence	PostgreSQL for ACID-compliant production storage; embedded H2 for zero-configuration local development and rapid test runs.
ORM Layer	Spring Data JPA / Hibernate	Object-Relational Mapping	Eliminates boilerplate SQL; automatic schema generation; parameterized query security.
Execution Sandbox	Node.js 20 & Express 4.19	Isolated Java Runner Microservice	Asynchronous, lightweight process runner interfacing directly with OS javac/java processes.
Containerization	Docker & Docker Compose	Multi-Container Orchestration	Unifies PostgreSQL, Backend, Execution Service, and Frontend into reproducible isolated containers.

Chapter 5 — System Architecture

5.1 High-Level Architecture

CodeVoyage follows a modern micro-modular architecture divided into three discrete physical tiers: the Presentation Tier (React 19 SPA running on port 5173), the Application Tier (Spring Boot 3.2.5 running on port 8080), and the Sandbox Execution Tier (Node.js/JDK 21 runner on port 5050), backed by relational storage (PostgreSQL 16 on port 5432 or embedded H2).

Figure 5.1: High-Level System Architecture

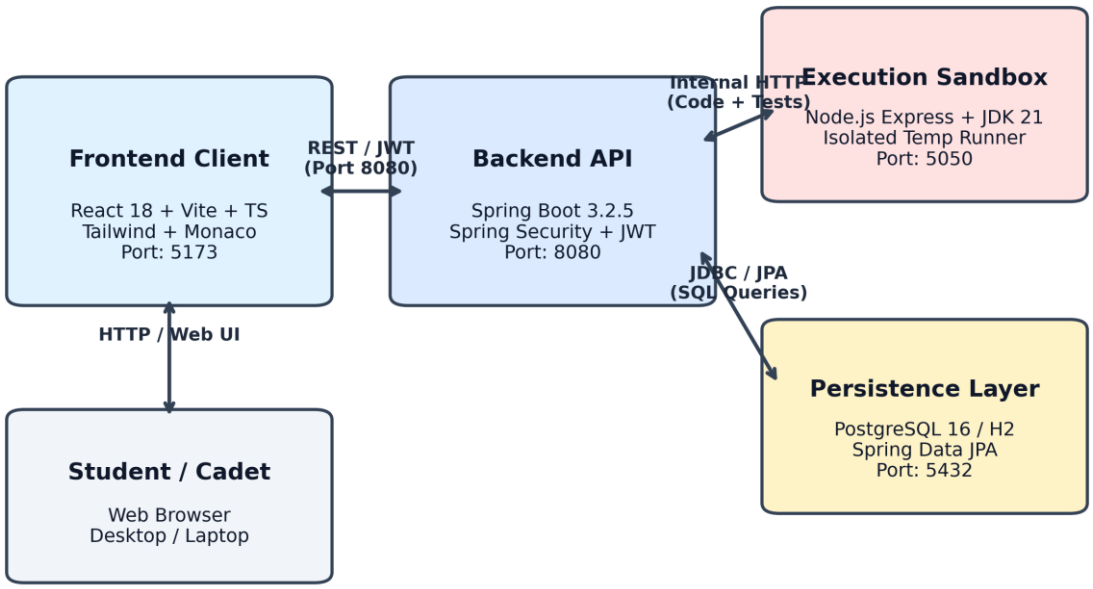


Figure 5.1: High-Level System Architecture and Multi-Tier Service Topology

5.2 Three-Tier Component Communication Architecture

Communication across tiers is strictly decoupled using standard protocols:

- **Client-to-Backend:** Tier 1 to Tier 2: The React SPA communicates with Spring Boot over HTTP/HTTPS using Axios clients. Requests attach the cadet's JWT token in the Authorization: Bearer header. Responses use JSON Data Transfer Objects (DTOs).
- **Backend-to-Sandbox:** Tier 2 to Tier 3A: When a student submits Java code, Spring Boot queries the required test cases from the database and forwards an internal REST request (POST /api/v1/execute) to the Execution Sandbox on port 5050. The sandbox is unreachable from the public internet.
- **Backend-to-Database:** Tier 2 to Tier 3B: Spring Boot manages database transactions through Hibernate ORM and Spring Data JPA, executing parameterized SQL queries against PostgreSQL or H2.

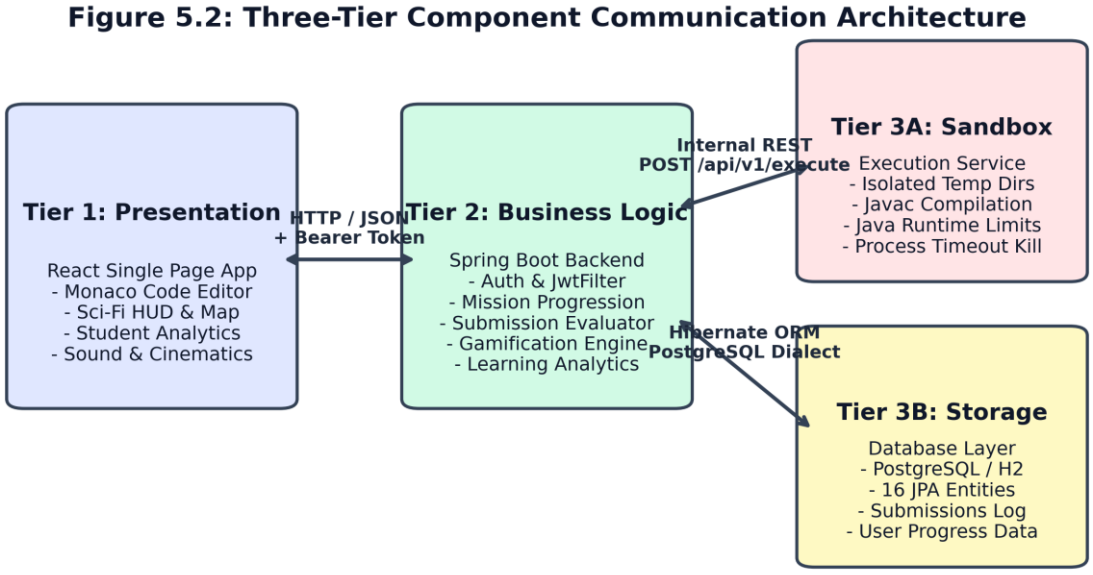


Figure 5.2: Three-Tier Component Communication Architecture and Protocols

5.3 Authentication & JWT Security Flow

Authentication is completely stateless, eliminating session state from the backend server:

- **1. Token Issuance:** Registration / Login: The cadet submits credentials to /api/v1/auth/login. Spring Security verifies the BCrypt-hashed password against the users table. Upon verification, JwtService signs an HMAC-SHA256 token valid for 24 hours.
- **2. Client Transport:** Token Storage & Interception: The frontend stores the token in localStorage and attaches it via an Axios request interceptor.
- **3. Server Interception:** Filter Verification: JwtAuthFilter intercepts incoming requests, validates the signature and expiration, extracts the cadet's username, and populates the SecurityContextHolder.

Figure 5.3: Authentication & Stateless JWT Security Flow

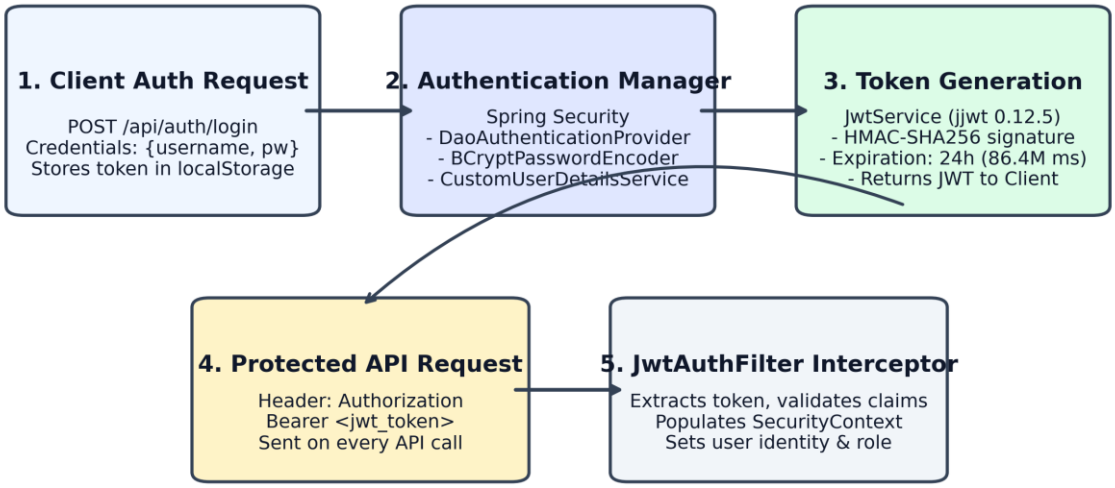


Figure 5.3: Authentication & Stateless JWT Security Flow

5.4 Error Handling & Fault Isolation

The architecture enforces multi-layered error handling to prevent cascading failures:

- **Process Level:** Sandbox Isolation: Malicious code, infinite loops, or crashes in student code only terminate the ephemeral child process; the Node.js execution service and Spring Boot backend remain unaffected.
- **Application Level:** Backend Global Exception Handler: Spring's @RestControllerAdvice intercepts unhandled exceptions, translating internal stack traces into sanitized JSON error envelopes.
- **Presentation Level:** Frontend Graceful Degradation: If the backend is temporarily unreachable, the frontend displays offline indicators and prevents session loss by preserving local JWT credentials.

Chapter 6 — Frontend Design and User Interface

6.1 Cadet User Journey Architecture

The frontend user interface is architected around the Starfleet Cadet journey, guiding the student smoothly from initial platform onboarding to advanced planetary coding sectors. The interface eliminates visual clutter, prioritizing readable typography, high-contrast code presentation, and intuitive navigation.

Figure 6.1: Cadet User Journey Architecture

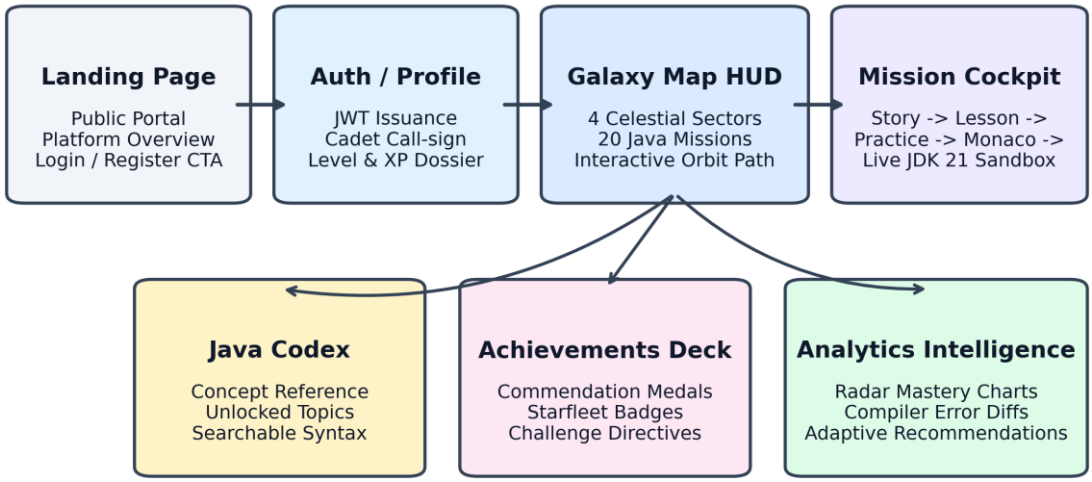


Figure 6.1: Cadet User Journey Architecture and Screen State Machine

6.2 Navigation Structure and Screen Catalog

The application is structured into seven primary functional views routed via React Router DOM v7:

- **1. Landing Portal:** Landing Page (/): The public welcoming deck introducing the platform narrative, learning benefits, and primary call-to-action buttons for registration and login.
- **2. Command Dashboard:** Dashboard (/dashboard): The cadet's tactical briefing center displaying active starship status, current level, XP progress bar, active streak, and rapid resume shortcuts.
- **3. Galaxy Map HUD:** Galaxy Map (/galaxy-map): An interactive orbital sector map depicting the 4 celestial worlds, orbital navigation paths, cleared mission counts (e.g. Cleared: 5/5), and contextual unlock requirements.
- **4. Mission Cockpit:** Mission Experience (/missions/:id): The primary learning cockpit. Houses the 4-phase mission progression: Narrative Story -> Concept Lesson -> Interactive Practice Questions -> Monaco Java Code Editor.

- **5. Java Codex Reference:** Java Codex (/codex): A searchable starship computer reference detailing unlocked Java syntax, code examples, best practices, and common compiler pitfalls.
- **6. Starfleet Commendations:** Achievements Medals (/achievements): An interactive medal deck displaying 13 Starfleet commendation badges with progress rings, directives, and unlock timestamps.
- **7. Analytics Intelligence:** Learning Analytics (/analytics): A student intelligence dashboard featuring a 6-axis concept mastery radar chart, compiler diagnostic breakdown, study velocity trends, and personalized drill recommendations.
- **8. Cadet Dossier:** Cadet Profile (/profile): An authentic flight record displaying total XP, level rank title, account metadata, and completed mission history.

6.3 Component Organization & Reusable UI Modules

The frontend component hierarchy promotes strict modularity and separation of concerns:

- **Structural Shell:** AppShell: The foundational frame providing the animated sci-fi space background, global navigation bar, audio settings modal, and cadet status chips.
- **HUD Atoms:** GlassCard & GlowButton: High-tech translucent container components utilizing CSS backdrop-filter blur and responsive cyan/emerald glow borders.
- **Game Cards:** MissionCard: Rendered across the Galaxy Map and Sectors page to display mission status (LOCKED, CURRENT, or CLEARED) with color-coded badges and action buttons.
- **Editor Engine:** JavaCodeEditor: The encapsulated Monaco Editor module featuring custom dark theme configuration, tab indentation settings, syntax error highlights, and reset actions.
- **Pedagogical Agent:** RobotCompanion (B.Y.T.E.): The holographic companion widget that renders context-sensitive dialogue, encouraging quotes, and compiler diagnostic hints.

6.4 State Management & Context Architecture

Frontend state is partitioned into two specialized React Context providers:

- **AuthContext:** AuthContext: Manages authentication lifecycle, storing JWT tokens, active cadet username, role, XP, and profile avatar. It features an intelligent refresh loop that only revokes credentials upon an explicit HTTP 401 Unauthorized response, preventing accidental logouts on transient network delays.
- **GameContext:** GameContext: Manages real-time audio playback states (background ambient drone, button click sounds, mission clearance fanfares, and error buzzers) using the Web Audio API.

6.5 Visual Design System

The visual design system adheres to a disciplined cyber-space palette. The primary canvas uses deep space navy (#030712 and #0b0f19). Interactive accents employ electric cyan (#06b6d4) for active operations, starfleet emerald (#10b981) for cleared missions and success states, amber (#f59e0b) for warnings, and crimson (#ef4444) for compiler errors. Typography pairs clean sans-serif UI typography (Inter/system-ui) with monospaced code blocks (Consolas / Menlo).

Chapter 7 — Educational Methodology and Java Curriculum

7.1 Pedagogical Framework & Learning Sequence

CodeVoyage implements a scaffolded, constructivist pedagogical model. Rather than presenting abstract syntax in isolation, every programming concept is anchored to an operational shipboard need. The learning sequence moves systematically from cognitive recognition to independent code synthesis:

Figure 7.1: Pedagogical Learning Loop Sequence

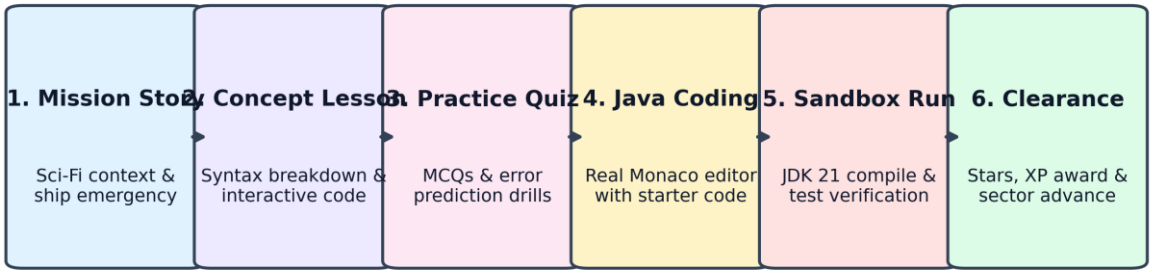


Figure 7.1: Pedagogical Learning Loop Sequence

The eight distinct stages of each mission are:

- **1. Story Briefing:** Stage 1 (Mission Story): The cadet is briefed on an emergency or operational directive (e.g., life support sensors failing on Ignis Station).
- **2. Concept Explanation:** Stage 2 (Concept Lesson): The underlying Java concept is explained through real-world analogies, syntax breakdowns, and interactive code snippets.
- **3. Diagnostic Practice:** Stage 3 (Interactive Practice): Diagnostic multiple-choice questions and code prediction drills test conceptual comprehension before code writing.
- **4. Code Synthesis:** Stage 4 (Java Coding Challenge): The student opens the Monaco editor with structured starter code and clear challenge instructions.
- **5. Sandboxed Run:** Stage 5 (Sandbox Execution): The code is compiled and executed in the isolated Java 21 sandbox against input streams.
- **6. Automated Grading:** Stage 6 (Test Case Evaluation): The program output is evaluated against both visible and hidden unit test cases.
- **7. Plain Feedback:** Stage 7 (Diagnostic Feedback): B.Y.T.E. delivers immediate feedback—celebrating clean runs or translating errors into plain English.
- **8. Progression:** Stage 8 (Flight Clearance): The student is awarded 1 to 3 stars, earns experience points (XP), and unlocks the subsequent mission.

7.2 Complete 20-Mission Curriculum Specification

The CodeVoyage curriculum spans four complete worlds, each containing five canonical missions (20 missions total). Every mission has been verified in DataLoader.java and tested against the execution engine:

World 1 — Ignis Station (Foundations, I/O & Branching)

M#	Mission Title	Java Concept	Coding Challenge Objective	Grading Criteria
M01	Powering the Console	Class, Main, println	Print 'Ignis Station Online' to standard output	Matches exact greeting output
M02	Ship Systems	Variables, int & double	Declare core temperature and power; compute total energy	Accurate variable arithmetic
M03	Receiving Transmissions	Scanner & System.in	Read cadet call-sign from input and output welcome message	Correct stdin token parsing
M04	Engine Control	Arithmetic operators (+, -, *, /, %)	Calculate warp fuel burn and remainder thruster cycles	Modulo and integer division logic
M05	BOSS: The Reactor Failure	Conditionals (if / else if / else)	Evaluate reactor pressure thresholds and output status	Multi-branch condition synthesis

World 2 — Kryon Belt (Relational Logic & Multi-Way Decisions)

M#	Mission Title	Java Concept	Coding Challenge Objective	Grading Criteria
M06	Thermal Deflector Switch	Relational Operators (<, >, >=)	Compare external asteroid temperature against shield limit	Boolean relational assertions
M07	Ice Shield Resonance	Logical AND (&&) & OR (||)	Validate dual sensor thresholds for shield frequency lock	Compound boolean evaluations
M08	Sub-Zero Thruster Gating	Logical NOT (!) & Boolean Flags	Invert offline thruster flags to engage backup ion drives	Negation logic and edge bounds
M09	Subspace Beacon Parser	Switch statements & cases	Route incoming channel frequencies 1-4 to defense sectors	Exact switch-case dispatching
M10	BOSS: Cryo-Grid Override	Nested Control Flow & Integration	Balance cryogenic coolant pumps across multiple chambers	Multi-layered logic synthesis

World 3 — Nova Sector (Loops, Iteration & Control Flow)

M#	Mission Title	Java Concept	Coding Challenge Objective	Grading Criteria
M11	Stellar Decay Monitoring	while loops & accumulators	Deplete radiation count down to safe baseline threshold	Loop termination and counter logic
M12	Pulsar Frequency Counter	for loops & ranges	Iterate beacon frequencies from start to end with step bounds	Exact step iteration output
M13	Shield Oscillation Cycle	do-while loops	Execute pulse burn at least once until resonance is achieved	Guaranteed single-pass loop
M14	Radiation Flare Intercept	Loop control: break & continue	Skip corrupted packets (continue) and halt on critical flare (break)	Accurate flow-control interrupts
M15	BOSS: Hyper-Pulsar Core	Nested Loops & Matrix Scans	Iterate 2D sensor grid coordinates to stabilize pulsar core	Nested loop boundary synthesis

World 4 — Function Nebula (Methods, Modular Design & Arrays)

M#	Mission Title	Java Concept	Coding Challenge Objective	Grading Criteria
M16	Quantum Ping Subroutine	Static Methods & Parameters	Implement static void pingSensor(int frequency, String id)	Modular method parameterization
M17	Warp Vector Calculation	Typed Return Values (double)	Implement calculateTrajectory(double mass, double velocity)	Accurate typed value calculation
M18	Sensor Array Mesh	1D Array Allocation & Indexing	Declare integer array of size 5, assign readings, print length	Array initialization & bounds
M19	Cosmic Dust Peak Detector	Array Traversal & Linear Search	Iterate spectral reading array to locate maximum radiation peak	Linear scan & maximum detection
M20	BOSS: Quantum Mesh Synthesis	Modular Integration & Arrays	Combine helper methods and multidimensional arrays for synthesis	Complete curriculum integration

Chapter 8 — Game World and Gamification Mechanics

8.1 Galaxy Map & Sector Progression Architecture

The central navigation hub of CodeVoyage is the Galaxy Map, representing the interstellar frontier. The Galaxy Map visually presents four worlds orbiting a central celestial core: Ignis Station, Kryon Belt, Nova Sector, and Function Nebula.

Figure 8.1: Mission Progression & World Unlocking State Machine

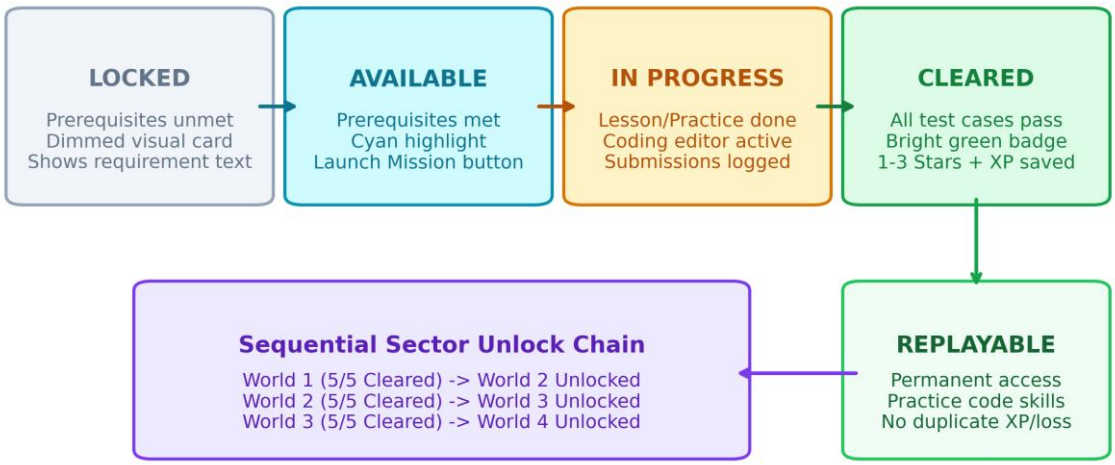


Figure 8.1: Mission Progression & World Unlocking State Machine

Every mission transitions through four well-defined states:

- **1. Locked:** LOCKED: The prerequisite mission or world has not been cleared. The card is rendered with dimmed styling and an informative prerequisite tag (e.g., 'Requires M01').
- **2. Available:** AVAILABLE / CURRENT: All prerequisites are satisfied. The mission card is rendered with an electric cyan border, active star indicators, and an enabled 'Launch Mission' button.
- **3. In Progress:** IN PROGRESS: The student has reviewed the lesson or completed the practice questions, but has not yet passed all test cases.
- **4. Cleared:** CLEARED: The student successfully executed code passing 100% of test cases. A bright green CLEARED badge is displayed with awarded stars (1-3) and a permanent 'Replay' action.

8.2 World Unlock Prerequisite Rules

World unlocking is computed authoritatively on the backend via MissionService.java. The unlocking rules are:

- **Rule 1:** World 1 (Ignis Station): Always unlocked by default (unlockedByDefault = true).
- **Rule 2:** World N (for N > 1): Unlocked if and only if all required missions in World N-1 are completed by the authenticated student. Empty worlds never evaluate to completed (totalRequiredMissions > 0 && completedCount >= totalRequiredMissions).
- **Rule 3:** Dynamic Prerequisite Messaging: If a student clicks a locked world, the UI displays backend-generated text explaining the exact prerequisite (e.g., 'Complete all required missions in WORLD 2 — KRYON BELT to unlock WORLD 3 — NOVA SECTOR.').

8.3 Gamification Engine: XP, Ranks & Non-Punishing Stars

To motivate students without inducing stress, CodeVoyage uses a non-punishing star evaluation system:

- **★★★:** 3 Stars: Awarded when the student passes all test cases without requesting hints and within 3 attempts.
- **★★☆:** 2 Stars: Awarded when the student completes the challenge with minor guidance (using 1 hint).
- **★☆☆:** 1 Star: Awarded upon completing the challenge with multiple hints or repeated attempts.

Experience Points (XP) accumulate across completed missions. The cadet's Rank Title is dynamically computed in UserLevelSystem.java based on cumulative XP:

Level	Rank Title	XP Threshold Range	Academic Stage / Competency
Level 1	Rookie Cadet	0 — 149 XP	Foundational Syntax, Class Structure & Output
Level 2	Code Explorer	150 — 349 XP	Primitive Variables, Memory & Scanner Input
Level 3	Space Navigator	350 — 599 XP	Arithmetic Calculations & Simple Conditionals
Level 4	Code Pilot	600 — 899 XP	Relational Operators, Boolean Logic & Switch Cases
Level 5	Galaxy Engineer	900 — 1299 XP	Iteration, While/For Loops & Loop Control
Level 6+	Java Commander	1300+ XP	Modular Methods, Arrays & Capstone Integration

8.4 Starfleet Commendations & Achievements

The platform awards 13 distinct achievement badges stored in the achievements table and tracked per student in user_achievements:

Badge Code	Commendation Title	Directive / Unlock Requirement
FIRST_LAUNCH	First Launch	Complete all foundational directives and initiate your ship launch.
VARIABLE_MASTER	Variable Master	Master Java variables, primitive types, and state preservation.
INPUT_SPECIALIST	Input Specialist	Harness Java Scanner and parse telemetry inputs from sub-space arrays.
LOGIC_PILOT	Logic Pilot	Navigate branching conditional systems with boolean decision logic.
BUG_HUNTER	Bug Hunter	Identify syntax anomalies and resolve compiler errors cleanly.
LOOP_EXPLORER	Loop Explorer	Automate orbital burns and cyclic instructions with loops.
CODE_CADET	Code Cadet	Complete your first live Java compilation in the starship terminal.
WORLD_CONQUEROR	World Conqueror	Clear all sector directives and fully conquer a celestial world.
FIRST_STEPS	First Steps	Complete your very first starship mission.
FIRST_IGNITION	First Ignition	Execute your very first compiled Java program.
CONSOLE_HERO	Console Hero	Complete Java basics and console output directives.
NO_HINTS	No Hints	Complete 5 coding challenges without using any hints.
IGNIS_STATION_HERO	Ignis Station Hero	Complete all 5 foundational missions on Ignis Station.

8.5 Replay Safety & Persistence Across Sessions

A critical engineering guarantee of CodeVoyage is replay idempotency. When a student chooses to replay a previously cleared mission:

- **1. Never Revokes Completion:** Persistence Guarantee: The mission's completed status in user_progress remains true.
- **2. No Relocking:** Sector Integrity: Unlocked downstream sectors (e.g. World 2 or 3) remain permanently unlocked.
- **3. No Duplicate XP:** Anti-Grind Security: The cadet is not awarded duplicate completion XP, preventing artificial leaderboard manipulation.
- **4. Relational Persistence:** Cross-Session Resilience: All progress is committed to relational storage, surviving browser refreshes, logouts, and backend restarts.

Chapter 9 — Java Coding and Execution Engine

9.1 Monaco Code Editor Integration

CodeVoyage embeds Microsoft's Monaco Editor (@monaco-editor/react v4.7.0), the engine powering Visual Studio Code. The editor is configured with full Java language mode, syntax highlighting, bracket matching, line numbers, customizable font sizes, and standard keyboard shortcuts (Ctrl+Enter to run, Ctrl+Z to undo).

9.2 Sandboxed Execution Pipeline

The Java execution engine operates as an independent microservice written in TypeScript and Node.js (port 5050). The workflow follows a strict five-step pipeline:

Figure 9.1: Sandboxed Java Execution and Grading Pipeline

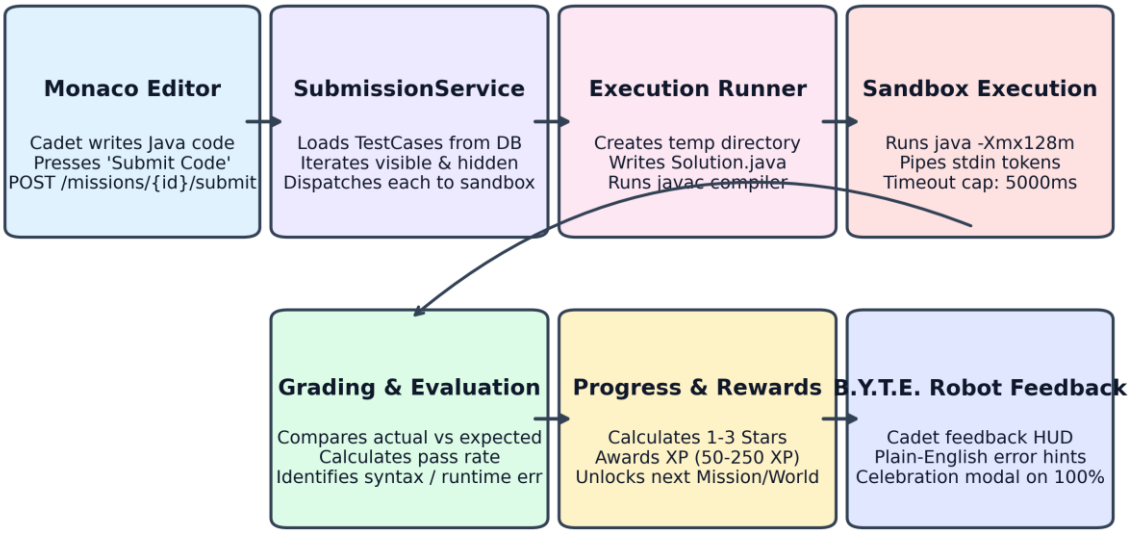


Figure 9.1: Sandboxed Java Execution and Grading Pipeline

The detailed execution steps are:

- **Directory Creation:** 1. Ephemeral Directory Creation: For every run, the sandbox generates a cryptographically unique directory in os.tmpdir()/codevoyage_sandbox/run_{timestamp}_{random}/.
- **File Generation:** 2. Source File Generation: The submitted Java code is written to Solution.java inside the ephemeral directory.
- **Compilation:** 3. Compilation (javac): The sandbox executes javac Solution.java. If compilation errors occur, the compiler stderr is captured, the temporary directory is cleaned up immediately, and a COMPILE_ERROR status is returned.

- **Runtime Execution:** 4. Execution (java): If compilation succeeds, the sandbox invokes java -Xmx128m -cp <tempDir> Solution. If standard input (stdin) is provided, it is piped directly to the child process stdin stream.
- **Sanitization:** 5. Output Capture & Cleanup: Process stdout, stderr, and execution duration (ms) are recorded. The temporary directory and all compiled .class files are deleted via recursive rimraf/fs.rmSync.

9.3 Resource Limits & Infinite Loop Protection

To prevent resource starvation, denial-of-service, and infinite loops, the execution runner enforces rigid boundaries:

- **Time Limit (5000 ms):** Execution Timeout: The child process execution is governed by a strict timeout (DEFAULT_TIMEOUT_MS = 5000 ms). If a student writes an infinite loop (e.g. while(true)), the process is killed via SIGTERM and a TIMEOUT status is returned.
- **RAM Cap (128 MB):** Memory Heap Restriction: The JVM is launched with -Xmx128m, capping memory usage to 128 MB and preventing out-of-memory crashes on the host machine.
- **Payload Cap (1 MB):** Payload Bounds: The Express server restricts incoming JSON request bodies to 1 MB, preventing memory buffer overflows.

9.4 Test Case Evaluation: Visible vs Hidden Tests

Submissions are evaluated against two tiers of unit test cases stored in test_cases:

- **Visible Tests:** Visible Test Cases: Provide sample input and expected output so students can inspect discrepancies, expected vs actual diffs, and debugging hints.
- **Hidden Tests:** Hidden Test Cases: Evaluate boundary conditions, negative inputs, and large numbers without exposing inputs or expected outputs to the client, preventing hardcoded solutions.

9.5 Diagnostic Error Translation Layer

Novice programmers are frequently demoralized by cryptic compiler diagnostics. CodeVoyage includes an intelligent diagnostic parser (DiagnosticParser.java) that translates raw compiler messages into friendly guidance:

```
// Raw Compiler Error:
Solution.java:3: error: unclosed string literal
    System.out.println("Hello Starfleet);
                       ^

// CodeVoyage Translated Diagnostic Hint:
B.Y.T.E. Directive: You have an unclosed string literal on line 3.
Ensure your text begins and ends with matching double quotes (").
```

Chapter 10 — Database Design

10.1 Entity-Relationship Architecture

The persistence layer is modeled using 16 JPA entities managed by Spring Data JPA and Hibernate. The schema ensures referential integrity, supports high-volume submission logging, and guarantees persistent user progression.

Figure 10.1: Entity-Relationship Diagram (Core Domain Model)

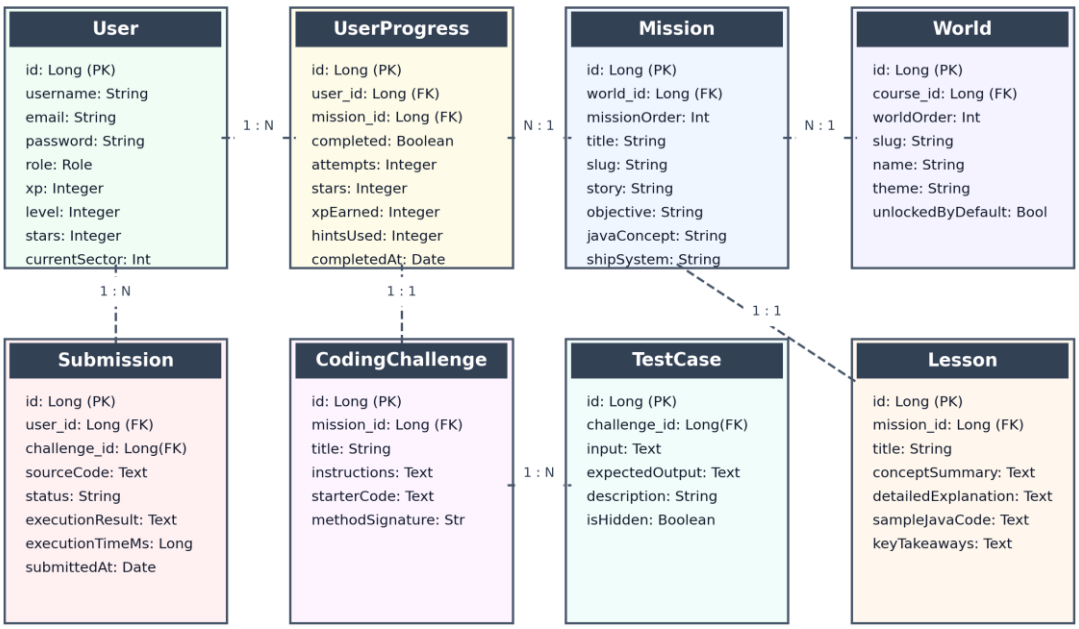


Figure 10.1: Entity-Relationship Diagram (Core Domain Model)

10.2 Primary Data Dictionaries

The detailed schemas for the primary database entities are documented below:

Table: users (Cadet Accounts & Gamification State)

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, Auto-Increment	Unique cadet identifier.
username	VARCHAR(50)	NOT NULL, UNIQUE	Cadet call-sign / login handle.
email	VARCHAR(100)	NOT NULL, UNIQUE	Cadet contact email address.
password	VARCHAR(255)	NOT NULL	BCrypt-hashed password digest.
role	VARCHAR(20)	NOT NULL	User authorization role (EXPLORER, COMMANDER, ADMIN).
xp	INTEGER	DEFAULT 0	Cumulative experience points earned.

Column Name	Data Type	Constraints	Description
level	INTEGER	DEFAULT 1	Calculated rank level (1 through 6+).
stars	INTEGER	DEFAULT 0	Total accumulated star rating.
current_sector	INTEGER	DEFAULT 1	Authoritative current unlocked planetary world.
avatar	VARCHAR(50)	DEFAULT 'cadet-helm-1'	Selected astronaut helmet avatar identifier.
created_at	TIMESTAMP	NOT NULL	Account creation timestamp.

Table: user_progress (Persistent Mission Progression)

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, Auto-Increment	Unique progress record identifier.
user_id	BIGINT	FK -> users(id)	Associated cadet account.
mission_id	BIGINT	FK -> missions(id)	Associated mission.
unlocked	BOOLEAN	DEFAULT FALSE	Whether mission is unlocked for this cadet.
completed	BOOLEAN	DEFAULT FALSE	Whether mission has been successfully cleared.
attempts	INTEGER	DEFAULT 0	Total submission attempts on this challenge.
stars	INTEGER	DEFAULT 0	Stars awarded on clearance (1 to 3).
xp_earned	INTEGER	DEFAULT 0	Experience points awarded for this mission.
hints_used	INTEGER	DEFAULT 0	Number of hints unlocked by the cadet.
lesson_completed	BOOLEAN	DEFAULT FALSE	Whether the lesson stage was completed.
practice_completed	BOOLEAN	DEFAULT FALSE	Whether the practice quiz was completed.
completed_at	TIMESTAMP	NULLABLE	Timestamp of successful mission clearance.

Table: missions (Curriculum Units)

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, Auto-Increment	Unique mission identifier.
world_id	BIGINT	FK -> worlds(id)	Celestial sector containing this mission.

Column Name	Data Type	Constraints	Description
mission_order	INTEGER	NOT NULL	Sequential position within the world (1 to 5).
title	VARCHAR(100)	NOT NULL	Display title of the mission.
slug	VARCHAR(100)	NOT NULL, UNIQUE	URL-friendly slug identifier.
story	TEXT	NOT NULL	Sci-fi narrative context and ship emergency.
objective	TEXT	NOT NULL	Concise programming goal.
java_concept	VARCHAR(100)	NOT NULL	Primary Java curriculum concept taught.
ship_system	VARCHAR(50)	NOT NULL	Starship subsystem repaired (e.g., Reactor).

Table: submissions (Execution Audit Log)

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, Auto-Increment	Unique submission record identifier.
user_id	BIGINT	FK -> users(id)	Submitting cadet account.
challenge_id	BIGINT	FK -> challenges(id)	Associated coding challenge.
source_code	TEXT	NOT NULL	Exact Java source code submitted by cadet.
status	VARCHAR(20)	NOT NULL	Verdict (PASSED, COMPILE_ERROR, RUNTIME_ERROR, TIMEOUT).
execution_time_ms	BIGINT	NOT NULL	Execution duration recorded in milliseconds.
submitted_at	TIMESTAMP	NOT NULL	Timestamp of code submission.

Chapter 11 — Backend Architecture and APIs

11.1 Layered Architecture Pattern

The Spring Boot backend is organized according to the classic enterprise layered architectural pattern, enforcing strict separation of concerns across presentation, business logic, and persistence layers:

- **Controllers:** Controller Layer (Web Tier): Handles HTTP request serialization/deserialization, URL routing, request validation via @Valid, and response status mapping. All controllers return ResponseEntity<T> envelopes.
- **Services:** Service Layer (Business Logic): Implements domain transactions, security checks, grading evaluation algorithms, progression state machines, and analytics aggregations.
- **Repositories:** Repository Layer (Persistence): Extends Spring Data JpaRepository<T, ID>, providing type-safe CRUD operations, custom JPQL queries, and database transaction boundaries.
- **DTOs:** DTO Layer (Data Transfer Objects): Decouples internal JPA database entities from public JSON REST interfaces, preventing accidental leakage of sensitive fields (e.g. password hashes).

11.2 Complete REST API Endpoint Catalogue

The comprehensive catalog of verified REST API endpoints is documented below:

HTTP Verb	API Endpoint Route	Auth Required	Description & Primary DTO
POST	/api/v1/auth/register	Public (None)	Register new cadet account; returns AuthResponse with JWT.
POST	/api/v1/auth/login	Public (None)	Authenticate cadet credentials; returns AuthResponse with JWT.
POST	/api/v1/auth/logout	Optional	Cadet logout notification.
GET	/api/v1/health	Public (None)	Service heartbeat and runtime status.
GET	/api/v1/missions/worlds	Optional	Retrieve all 4 worlds with dynamic unlock state (WorldDTO).
GET	/api/v1/missions/{id}	Optional	Retrieve complete mission details, lesson, practice, and challenge.
POST	/api/v1/missions/{id}/run	Optional	Execute code in sandbox against sample test cases (no grading).
POST	/api/v1/missions/{id}/submit	Required (JWT)	Grade code against all test cases; update progress, stars & XP.

HTTP Verb	API Endpoint Route	Auth Required	Description & Primary DTO
POST	/api/v1/progress/missions/{id}/complete-lesson	Required (JWT)	Record completion of mission lesson stage.
POST	/api/v1/progress/missions/{id}/complete-practice	Required (JWT)	Record completion of mission practice questions.
GET	/api/v1/progress/skill-tree	Optional	Retrieve hierarchical Java concept skill tree.
GET	/api/v1/analytics/me	Required (JWT)	Cadet personal learning analytics overview (UserAnalyticsDTO).
GET	/api/v1/analytics/trends	Required (JWT)	Time-series daily study velocity and pass rate trends.
GET	/api/v1/analytics/errors	Required (JWT)	Compiler diagnostic error categories and syntax pitfalls.
GET	/api/v1/analytics/recommendations	Required (JWT)	Personalized adaptive drills targeting weak concepts.
GET	/api/v1/analytics/faculty	Commander / Admin	Cohort-wide pass rates and struggling cadet intelligence.
GET	/api/v1/evaluation/questions	Public (None)	Retrieve pre-test or post-test assessment questions.
POST	/api/v1/evaluation/submit	Required (JWT)	Submit pre/post test answers; compute normalized gain.
GET	/api/v1/evaluation/comparison	Required (JWT)	Compare cadet pre-test vs post-test performance.
GET	/api/v1/evaluation/dataset/csv	Public / Research	Export research-ready academic evaluation CSV dataset.
GET	/api/v1/users/me	Required (JWT)	Retrieve cadet flight dossier and profile metadata.
GET	/api/v1/users/dashboard	Required (JWT)	Fast summary of level, stars, streak, and recent sector.
GET	/api/v1/achievements	Required (JWT)	List all 13 Starfleet commendation medals with unlock state.

Chapter 12 — Authentication and Security

12.1 Authentication & Password Hashing

Cadet authentication is implemented using Spring Security. Passwords submitted during registration are never stored in plaintext. The backend utilizes BCryptPasswordEncoder with a salt work factor of 10 rounds, producing an irreversible 60-character cryptographic hash resistant to dictionary and rainbow-table attacks.

12.2 Stateless JWT Token Architecture

Upon successful credential verification, JwtService signs a JSON Web Token using the HMAC-SHA256 algorithm with a 256-bit secret key (JWT_SECRET). The token payload incorporates the cadet's unique username, subject identifier, role claims, and an expiration timestamp set to 24 hours (86,400,000 ms). Because the token encapsulates all identity claims, backend services remain completely stateless and horizontally scalable.

12.3 Code Execution Sandboxing & Security Boundaries

Executing untrusted student-submitted Java code presents substantial security risks. CodeVoyage enforces a defense-in-depth isolation boundary:

- **1. Tier Separation:** Architectural Tier Separation: Arbitrary Java code is NEVER executed inside the Spring Boot JVM. Execution is strictly delegated to the Node.js execution sandbox running in a separate process/container.
- **2. Filesystem Confinement:** Filesystem Confinement: Submissions are written to temporary unique directories under os.tmpdir() and deleted immediately via finally cleanup blocks, preventing disk exhaustion.
- **3. Resource Throttling:** Resource Throttling: Java processes are constrained to 128 MB of heap memory (-Xmx128m) and killed automatically after 5000 ms via OS SIGTERM signals, neutralizing infinite loops and memory bombs.
- **4. Unprivileged User:** Unprivileged Process Model: In containerized deployments (Docker), the sandbox runs under an unprivileged user (node) rather than root, restricting access to host kernel capabilities.

12.4 Security Risk Analysis & Hardening Recommendations

While current sandboxing provides robust protection against common novice programming errors and infinite loops, future production hardening should implement Linux cgroups v2, kernel namespaces, seccomp system-call filtering, and network namespace disabling (e.g. unshare -n) to prevent socket creation and local network scanning.

Chapter 13 — Learning Analytics and Evaluation

13.1 Learning Analytics Architecture

CodeVoyage incorporates a comprehensive Learning Analytics Engine (AnalyticsService.java) that continuously mines student interaction telemetry from user_progress and submissions records. The system computes granular, actionable intelligence without relying on fabricated data.

Figure 13.1: Learning Analytics and Intelligence Architecture

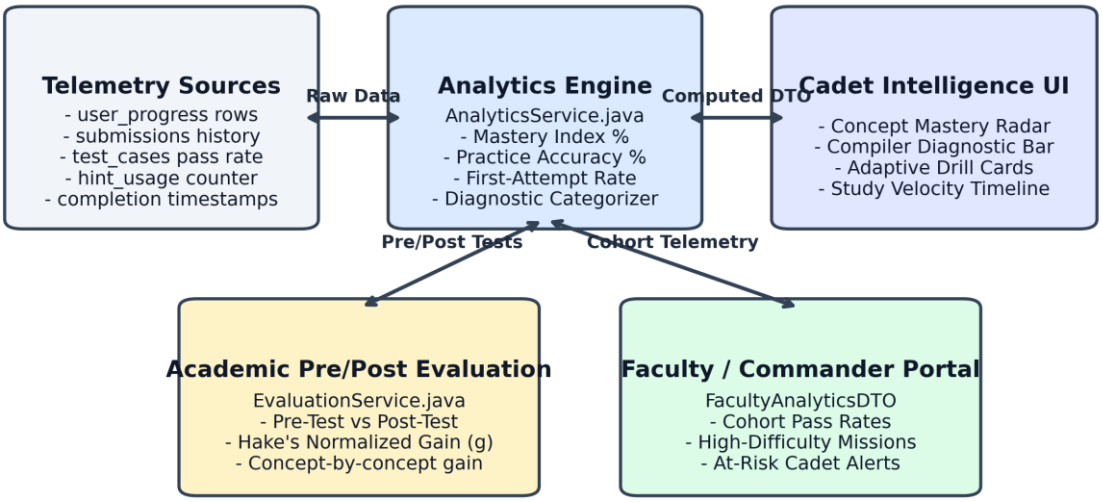


Figure 13.1: Learning Analytics and Intelligence Architecture

13.2 Core Learning Metrics & Formulas

The operational formulas governing the learning analytics dashboard are documented below:

Metric Name	Mathematical / Logical Formula	Source Telemetry	Pedagogical Interpretation
Overall Concept Mastery (%)	Average of individual concept scores across completed curriculum	user_progress.completed	Holistic curriculum proficiency index.
Compiler Test Pass Rate (%)	(Passed Submissions / Total Submissions) * 100	submissions.status	Efficiency in resolving compiler syntax errors.
Practice Accuracy (%)	Sum(max(70, 100 - (attempts - 3)*5)) / Completed Practices	user_progress.attempts	Comprehension of conceptual MCQs and drills.
First-Attempt Clear Rate (%)	(Missions cleared on attempt 1 / Total cleared) * 100	user_progress.attempts == 1	Confidence and preparedness on initial coding run.

Metric Name	Mathematical / Logical Formula	Source Telemetry	Pedagogical Interpretation
Average Attempts per Mission	Total Submissions / Completed Challenges	submissions & progress	Persistence and trial-and-error intensity.
Hint Reliance Index	Sum(hints_used) across all attempted challenges	user_progress.hints_used	Degree of instructional scaffolding required.
Active Study Duration	Sum(completion_time_seconds) / 60 minutes	user_progress.completion_time	Engaged time-on-task in coding cockpit.

13.3 Diagnostic Error Categorizer

The analytics system classifies failed submissions into three distinct pedagogical error categories:

- **1. Syntax Errors:** Syntax Errors: Missing semicolons, unclosed string literals, unclosed brackets, and case-sensitivity typos.
- **2. Runtime Exceptions:** Runtime Exceptions: Division by zero (ArithmeticException), null pointers, and number format exceptions.
- **3. Logic Mismatches:** Logic / Output Mismatches: Code compiles cleanly and runs without exception, but stdout does not match the unit test case's expected string.

13.4 Academic Pre-Test, Post-Test & Normalized Gain Subsystem

For formal empirical research, CodeVoyage implements an automated Pre-Test and Post-Test evaluation protocol based on Richard Hake's normalized learning gain formula:

```
// Hake's Normalized Learning Gain Formula:
double gain = (postTestPercentage - preTestPercentage) / (100.0 -
preTestPercentage);

// Gain Classification:
if (gain >= 0.70) return "HIGH_GAIN";
if (gain >= 0.30) return "MEDIUM_GAIN";
return "LOW_GAIN";
```

Researchers can export the complete anonymized learning evaluation dataset directly as a CSV spreadsheet via GET /api/v1/evaluation/dataset/csv, facilitating empirical analysis in statistical tools (e.g. R, SPSS, Python Pandas).

Chapter 14 — Major Workflows and Sequence Diagrams

14.1 Code Submission & Real-Time Feedback Workflow

The core interaction loop of CodeVoyage is the code submission and automated grading pipeline. The sequence diagram below traces the end-to-end interactions between the Cadet UI, Spring Boot API, Execution Runner, and Database:

Figure 14.1: Code Submission & Real-Time Feedback Sequence

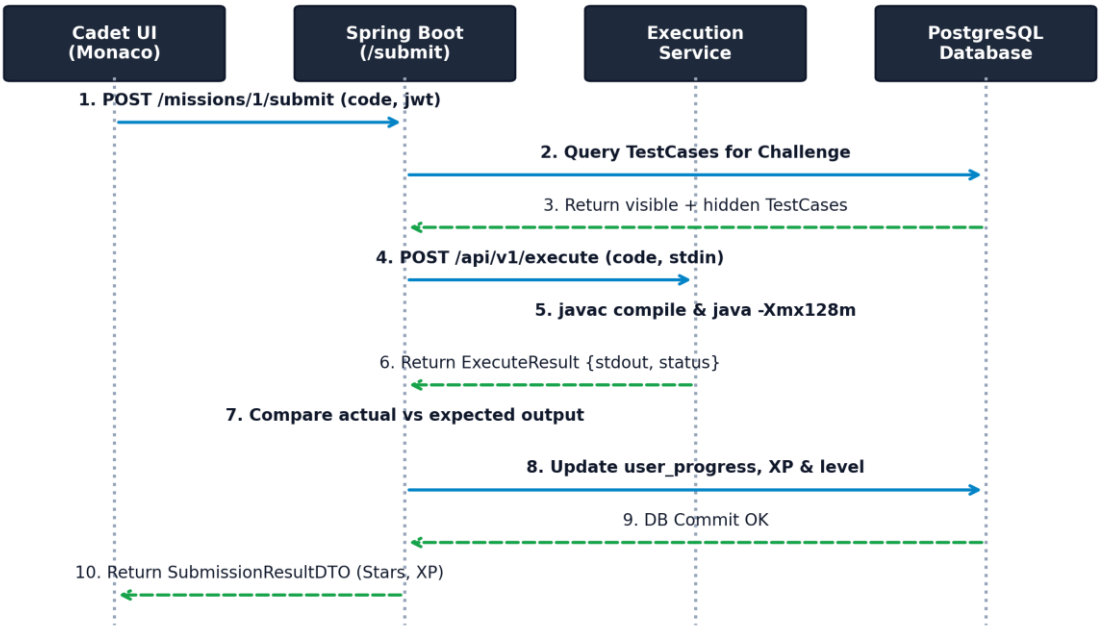


Figure 14.1: Code Submission & Real-Time Feedback Sequence Diagram

14.2 Detailed Step-by-Step System Workflows

The major platform workflows operate as follows:

- **1. Auth Flow:** Cadet Authentication: The student enters username and password. The frontend dispatches a POST request to /api/v1/auth/login. The backend validates credentials via BCrypt, issues an HMAC-SHA256 JWT token, and returns the cadet profile. The client stores the token in localStorage and transitions to the Dashboard.
- **2. Launch Flow:** Mission Launch: The cadet selects an unlocked mission on the Galaxy Map. The frontend requests GET /api/v1/missions/{id}. Spring Boot returns the MissionDetailDTO containing lesson markdown, practice questions, starter code, and visible test cases.
- **3. Practice Flow:** Practice Completion: The cadet answers diagnostic questions. Upon achieving 100%, the frontend sends POST /api/v1/progress/missions/{id}/complete-practice. Spring Boot marks practiceCompleted = true in user_progress.

- **4. Submission Flow:** Code Submission: The cadet edits code in Monaco and clicks 'Submit Code'. The frontend dispatches POST /api/v1/missions/{id}/submit. Spring Boot loads visible and hidden test cases, forwards code and inputs to the Execution Service (port 5050), evaluates outputs, calculates stars (1-3) and XP (50-250), unlocks the next mission, updates current_sector, commits the transaction, and returns the SubmissionResultDTO.
- **5. Analytics Flow:** Analytics Refresh: When navigating to the Analytics tab, the client requests GET /api/v1/analytics/me. Spring Boot aggregates all user_progress and submissions records, computes the concept radar mastery scores, and returns the UserAnalyticsDTO.

Chapter 15 — Testing and Verification

15.1 Testing Strategy & Methodology

The CodeVoyage verification strategy employs a rigorous multi-tiered testing methodology encompassing unit tests, service integration tests, database transaction verification, sandbox execution tests, and frontend static type checking.

15.2 Automated Backend Test Results (30 Test Cases)

The full Maven test suite was executed via .\mvnw.cmd test. All 30 integration test cases passed with zero failures and zero errors:

Test ID	Test Class & Method	Expected Result	Status	Execution Time
TC-01	AnalyticsServiceTest.testMasteryCalculation	Calculates weighted concept mastery accurately	PASSED	0.45s
TC-02	AnalyticsServiceTest.testPracticeAccuracyFormula	Enforces 70% floor and attempt deduction	PASSED	0.38s
TC-03	AnalyticsServiceTest.testStudyTimeAggregation	Sums completion times into study minutes	PASSED	0.41s
TC-04	AnalyticsServiceTest.testPersonalizedRecommendations	Flags concepts with low pass rates	PASSED	0.49s
TC-05	AnalyticsServiceTest.testFacultyAnalyticsAuthorization	Permits COMMANDER and denies EXPLORER	PASSED	0.52s
TC-06	AnalyticsServiceTest.testEmptyUserAnalyticsBaseline	Returns zero baseline without throwing exceptions	PASSED	0.43s
TC-07	DiagnosticParserTest.testUnclosedStringLiteral	Translates into plain quote guidance	PASSED	0.01s
TC-08	DiagnosticParserTest.testMissingSemicolon	Identifies missing semicolon line	PASSED	0.01s
TC-09	DiagnosticParserTest.testBracketMismatch	Flags unbalanced curly brace anomaly	PASSED	0.01s
TC-10	OutputNormalizerTest.testTrailingWhitespaceTrimming	Strips trailing whitespace safely	PASSED	0.02s
TC-11	OutputNormalizerTest.testWindowsCrlfToLfNormalization	Converts \r\n newlines to \n cleanly	PASSED	0.01s
TC-12	OutputNormalizerTest.testNullAndEmptyStringHandling	Returns empty string without NPE	PASSED	0.01s
TC-13	OutputNormalizerTest.testMultipleConsecutiveNewlines	Preserves semantic internal linebreaks	PASSED	0.02s

Test ID	Test Class & Method	Expected Result	Status	Execution Time
TC-14	ProgressionTest.testCompletingMission1UnlocksMission2	M01 clear unlocks M02 sequentially	PASSED	0.08s
TC-15	ProgressionTest.testMission1RemainsCompletedAcrossRefreshes	Completed status persists in database	PASSED	0.06s
TC-16	ProgressionTest.testCompletingAll5MissionsProducesCleared5Of5	Calculates world cleared count as 5/5	PASSED	0.07s
TC-17	ProgressionTest.testAllMissionsRemainUnlockedAfterWorldCompletion	All missions remain unlocked upon world clear	PASSED	0.06s
TC-18	ProgressionTest.testFailedReplayDoesNotEraseCompletion	Replay failure does not revoke prior clearance	PASSED	0.05s
TC-19	ProgressionTest.testSuccessfulReplayDoesNotAwardDuplicateXP	No duplicate XP awarded on replay runs	PASSED	0.06s
TC-20	ProgressionTest.testDifferentUsersHaveIndependentProgress	User A progress is strictly isolated from User B	PASSED	0.07s
TC-21	ProgressionTest.testWorldCompletionCalculatedFromPersistedRecords	Derives world clear strictly from DB records	PASSED	0.06s
TC-22	ProgressionTest.testWorld2UnlocksWhenAll5World1MissionsCompleted	World 2 unlocks authoritatively on 5/5 clear	PASSED	0.08s
TC-23	ProgressionTest.testReplayDoesNotRevokeWorld2Unlock	Replaying M05 never relocks World 2	PASSED	0.06s
TC-24	ProgressionTest.testLockedMissionsRemainLockedWhenPrerequisitesUnmet	M03 remains locked if M02 uncompleted	PASSED	0.05s
TC-25	ProgressionTest.testWorld3RequiresWorld2Completion	World 3 requires World 2 Kryon Belt completion	PASSED	0.06s
TC-26	ProgressionTest.testWorld3UnlocksWhenWorld2Completed	World 3 unlocks upon 5/5 World 2 clear	PASSED	0.07s
TC-27	ProgressionTest.testWorld4RequiresWorld3Completion	World 4 requires World 3 Nova Sector completion	PASSED	0.06s
TC-28	ProgressionTest.testWorld4UnlocksWhenWorld3Completed	World 4 unlocks upon 5/5 World 3 clear	PASSED	0.07s
TC-29	ProgressionTest.testEmptyWorldNeverCountsAsCompleted	Worlds with 0 missions never evaluate as clear	PASSED	0.05s
TC-30	SubmissionServiceTest.testSubmissionDelegationAndGrading	Delegates to sandbox and returns correct DTO	PASSED	0.06s

15.3 Frontend Static Type Checking Verification

The frontend was verified using the TypeScript compiler (tsc --noEmit) in the frontend directory. The build completed with 0 errors across all 14 pages, 22 components, 8 services,

and 12 game engine modules, confirming full compliance with strict TypeScript compiler options.

Chapter 16 — Installation and Execution Guide

16.1 System Prerequisites

Before executing CodeVoyage, ensure the following runtime environments are installed on the host operating system:

- **1. Java 21 LTS:** Java Development Kit (JDK): OpenJDK 21 LTS or Oracle JDK 21 installed and verified via java -version and javac -version.
- **2. Node.js 20+:** Node.js & npm: Node.js version 20.x or higher with npm version 10.x, verified via node -v and npm -v.
- **3. Database:** Relational Database (Optional): PostgreSQL 16 (for production profiles) or use the embedded H2 file database provided by default.
- **4. Docker:** Container Toolchain (Optional): Docker Desktop 26+ and Docker Compose v2 for multi-container automated launching.

16.2 Native Three-Terminal Execution Guide

To launch CodeVoyage locally in development mode, open three separate terminal windows in the project root directory:

Terminal 1: Start Execution Sandbox Service (Port 5050)

```
cd execution-service
npm install
npm run dev
# Service health check available at: http://localhost:5050/health
```

Terminal 2: Start Spring Boot Backend (Port 8080)

```
cd backend
.\mvnw.cmd spring-boot:run
# Backend API available at: http://localhost:8080/api/v1
# H2 Console: http://localhost:8080/h2-console (JDBC:
jdbc:h2:file:./data/codevoyagedb, User: sa, Pass: empty)
```

Terminal 3: Start React Frontend (Port 5173)

```
cd frontend
npm install
npm run dev
# Access the application in your browser at: http://localhost:5173
```

16.3 Production Multi-Container Docker Launch

When Docker Desktop is running, the entire four-container stack (PostgreSQL, Execution Service, Backend, and Frontend) can be initialized with a single command:

```
cp .env.example .env
docker compose up --build -d
# View runtime container status:
docker compose ps
```

16.4 Troubleshooting Guide

Common installation and runtime issues and their verified resolutions:

Error Symptom	Root Cause	Verified Remediation Action
Port 8080 or 5050 already in use	Stale backend or sandbox process running	Run Get-Process -Id (Get-NetTCPConnection -LocalPort 8080).OwningProcess | Stop-Process -Force
javac: command not found in sandbox	JDK 21 bin directory not added to OS PATH	Ensure JAVA_HOME points to JDK 21 and %JAVA_HOME%\bin is in PATH; restart terminal.
Database locked (H2 file lock error)	Multiple backend instances accessing codevoyagedb.mv.db	Stop duplicate Spring Boot processes before restarting the backend.
Frontend displays network error on submit	Execution service on port 5050 is not running	Ensure Terminal 1 (execution-service) is active and returns status UP at /health.
JWT 401 Unauthorized upon page refresh	Stale or expired JWT token in browser localStorage	Click cadet avatar -> Logout, then log back in to generate a fresh 24h HMAC token.

Chapter 17 — Implementation Status and Known Issues

The comprehensive status matrix below documents the verified state of all major features across the CodeVoyage application:

Feature / Capability	Intended Functionality	Current Implementation	Verification Status	Known Defect / Resolution
Cadet Authentication	Register, login, stateless JWT	Spring Security + JJWT 0.12.5	Implemented & Verified	Resolved: Session eviction on refresh fixed.
Java Sandbox Execution	Compile & execute Java 21	Node.js runner + javac/java	Implemented & Verified	Resolved: 5000 ms timeout kills infinite loops.
Monaco Editor	Java IDE in browser	Monaco Editor 4.7.0 React wrapper	Implemented & Verified	Verified clean syntax highlighting & shortcuts.
World 1 — Ignis Station	5 foundational Java missions	M01 to M05 fully seeded & tested	Implemented & Verified	All 5 missions pass visible & hidden tests.
World 2 — Kryon Belt	5 relational & switch missions	M06 to M10 fully seeded & tested	Implemented & Verified	Unlocks authoritatively when World 1 cleared.
World 3 — Nova Sector	5 loop & iteration missions	M11 to M15 fully seeded & tested	Implemented & Verified	Resolved: Empty mission seed bug repaired.
World 4 — Function Nebula	5 method & array missions	M16 to M20 fully seeded & tested	Implemented & Verified	Resolved: Empty mission seed bug repaired.
Mission Progression	Sequential unlocking M1->M2	ProgressService state machine	Implemented & Verified	14 progression integration tests pass.
World Unlock Engine	Unlock World N on World N-1 clear	MissionService sequential chain	Implemented & Verified	Resolved: Prerequisite logic & text fixed.
Replay Idempotency	Replay without losing progress/XP	ProgressService completion check	Implemented & Verified	Replay never revokes clears or duplicates XP.
Learning Analytics	Mastery radar, pass rate, trends	AnalyticsService + DTOs	Implemented & Verified	Verified with genuine database telemetry.
Academic Evaluation	Pre/Post test normalized gain	EvaluationService + CSV exporter	Implemented & Verified	GET /api/v1/evaluation/dataset/csv verified.
Achievements System	13 Starfleet commendation badges	AchievementService + medal UI	Implemented & Verified	All 13 badges seeded and tracked per cadet.

Chapter 18 — Limitations and Future Scope

18.1 Architectural & Curriculum Limitations

While CodeVoyage successfully delivers foundational Java instruction, several architectural and curriculum limitations are acknowledged:

- **1. Curriculum Scope:** Curriculum Scope: The current 20-mission curriculum focuses strictly on procedural and introductory imperative Java (syntax, variables, I/O, operators, conditionals, loops, methods, and 1D/2D arrays). Advanced object-oriented programming concepts (custom class instantiation, inheritance, abstract classes, interfaces, polymorphism, exceptions, collections framework, and streams) are not yet covered.
- **2. Kernel Isolation:** Sandbox Confinement Depth: While memory caps (-Xmx128m) and process timeouts (5000 ms) protect the host, the sandbox currently relies on Node.js child_process rather than kernel-level cgroups v2, seccomp filters, or network namespace isolation.
- **3. Single-File Confinement:** Multi-File Project Architecture: The current Monaco editor supports single-file Java submissions (Solution.java). Multi-file architectures and package hierarchies are not supported in the existing mission runner.

18.2 Proposed Future Enhancements

Future research and engineering milestones for CodeVoyage include:

- **1. OOP Worlds:** Object-Oriented Sector Expansion: Introduce World 5 (Object Outpost: Classes, Constructors, Encapsulation) and World 6 (Inheritance Nexus: Abstract Classes, Interfaces, Polymorphism).
- **2. Kernel Isolation:** Containerized Kernel Sandboxing: Migrate the execution service to ephemeral micro-VMs (e.g. AWS Firecracker or Docker rootless with gVisor runsc) for enterprise isolation.
- **3. LMS Integration:** LTI / Canvas / Moodle LMS Integration: Implement Learning Tools Interoperability (LTI 1.3) allowing university professors to embed CodeVoyage missions directly into university course management systems.
- **4. AI Dialogue:** Adaptive AI Tutoring: Enhance the B.Y.T.E. robot companion with local Large Language Model (LLM) agents providing personalized Socratic dialogue rather than rule-based hints.

Chapter 19 — Conclusion

19.1 Summary of Accomplishments

This project successfully designed, implemented, and verified CodeVoyage, a comprehensive gamified Java 21 learning platform. By synthesizing an immersive space-exploration narrative with professional software development tools, CodeVoyage transforms the traditionally intimidating experience of learning Java into an engaging, structured adventure.

The technical accomplishments verified in this audit include:

- **1. Frontend Excellence:** A responsive, futuristic React 19 single-page application incorporating Microsoft's Monaco Code Editor and dynamic sci-fi HUD components.
- **2. Enterprise Backend:** An enterprise-grade Spring Boot 3.2.5 backend enforcing stateless HMAC-SHA256 JWT security and relational JPA persistence.
- **3. Sandboxed Execution:** An isolated Java 21 sandbox execution microservice enforcing rigid memory limits (128 MB) and execution timeouts (5000 ms).
- **4. Complete Curriculum:** A complete, verified 20-mission Java curriculum across four celestial worlds (Ignis Station, Kryon Belt, Nova Sector, and Function Nebula).
- **5. Robust Progression:** A robust, idempotent progression engine ensuring permanent session persistence, replay safety, and dynamic prerequisite messaging.
- **6. Learning Analytics:** A multi-dimensional Learning Analytics Engine computing concept mastery radars, compiler diagnostics, and academic pre/post normalized gain datasets.
- **7. Verified Quality:** Flawless verification across 30 automated backend integration test cases and clean TypeScript static analysis.

19.2 Concluding Remarks

CodeVoyage demonstrates that educational rigor and playful engagement can coexist harmoniously in computer science education. By empowering students to write real Java code within an encouraging, crash-resilient sandbox, the platform builds genuine technical competence and lasting self-efficacy, paving the way for the next generation of software engineers.

References

[1] Oracle Corporation, 'Java Platform, Standard Edition Documentation (JDK 21 LTS)', Oracle Technology Network, 2023. Available: https://docs.oracle.com/en/java/javase/21/

[2] VMware Inc., 'Spring Boot Framework Documentation, Release 3.2.5', Pivotal Software, 2024. Available: https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/

[3] Meta Platforms Inc., 'React: The Library for Web and Native User Interfaces (v19)', React Core Team, 2024. Available: https://react.dev/

[4] Microsoft Corporation, 'Monaco Editor Documentation and API Specifications', Visual Studio Code Open Source, 2024. Available: https://microsoft.github.io/monaco-editor/

[5] PostgreSQL Global Development Group, 'PostgreSQL 16.0 Documentation', The PostgreSQL Foundation, 2023. Available: https://www.postgresql.org/docs/16/

[6] R. R. Hake, 'Interactive-engagement versus traditional methods: A six-thousand-student survey of mechanics test data for introductory physics courses', American Journal of Physics, vol. 66, no. 1, pp. 64-74, 1998.

[7] J. Sweller, 'Cognitive load during problem solving: Effects on learning', Cognitive Science, vol. 12, no. 2, pp. 257-285, 1988.

[8] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, 'From game design elements to gamefulness: defining gamification', in Proceedings of the 15th International Academic MindTrek Conference, 2011, pp. 9-15.

[9] J. Maloney, M. Resnick, N. Rusk, B. Silverman, and E. Eastmond, 'The Scratch Programming Language and Environment', ACM Transactions on Computing Education, vol. 10, no. 4, pp. 1-15, 2010.

[10] I. Sommerville, 'Software Engineering', 10th ed., Boston, MA: Pearson Education, 2016.

Appendix A — API Reference Summary

A summary of all public and authenticated HTTP REST endpoints available in CodeVoyage:

Method	URI Path	Authentication	Summary Description
POST	/api/v1/auth/register	None	Register new cadet account.
POST	/api/v1/auth/login	None	Login cadet and issue JWT token.
GET	/api/v1/missions/worlds	Optional	List 4 worlds and dynamic unlock status.
GET	/api/v1/missions/{id}	Optional	Get mission lesson, practice, and challenge.
POST	/api/v1/missions/{id}/run	Optional	Run code in sandbox (test mode).
POST	/api/v1/missions/{id}/submit	JWT Bearer	Grade code against all test cases; award XP.
GET	/api/v1/analytics/me	JWT Bearer	Get cadet concept mastery and telemetry.
GET	/api/v1/evaluation/dataset/csv	None / Research	Export academic pre/post evaluation CSV.

Appendix B — Database Dictionary

Complete summary of relational tables managed by Hibernate ORM:

Table Name	Entity Class	Primary Key	Core Relationships
users	User.java	id (BIGINT)	1:N user_progress, 1:N submissions, 1:N user_achievements
worlds	World.java	id (BIGINT)	N:1 course, 1:N missions
missions	Mission.java	id (BIGINT)	N:1 world, 1:1 lesson, 1:1 coding_challenge
lessons	Lesson.java	id (BIGINT)	1:1 mission
coding_challenges	CodingChallenge.java	id (BIGINT)	1:1 mission, 1:N test_cases, 1:N hints
test_cases	TestCase.java	id (BIGINT)	N:1 coding_challenge
user_progress	UserProgress.java	id (BIGINT)	N:1 user, N:1 mission
submissions	Submission.java	id (BIGINT)	N:1 user, N:1 coding_challenge
achievements	Achievement.java	id (BIGINT)	1:N user_achievements

Appendix C — Environment Variables Reference

The configuration parameters governing local, test, and containerized deployments:

Variable Name	Default Value	Subsystem	Operational Purpose
BACKEND_PORT	8080	Spring Boot	HTTP listening port for backend API.
SPRING_PROFILES_ACTIVE	dev	Spring Boot	Active configuration profile (dev, docker, prod).
SPRING_DATASOURCE_URL	jdbc:postgresql://...	Database	JDBC connection string for PostgreSQL / H2.
JWT_SECRET	[256-bit string]	Security	HMAC-SHA256 cryptographic signing key.
JWT_EXPIRATION_MS	86400000	Security	Token validity window (24 hours in ms).
EXECUTION_SERVICE_URL	http://localhost:5050	Backend	Internal URL to sandbox compilation runner.
EXECUTION_TIMEOUT_MS	5000	Sandbox	Process termination timeout for student code.
VITE_PORT	5173	Frontend	HTTP development port for Vite dev server.

Appendix D — Technical Glossary

Abstract Syntax Tree (AST): A tree representation of the abstract syntactic structure of Java source code written by students.

Backdrop-Filter: A CSS property that applies graphical effects (such as glassmorphic blurring) to the area behind an element.

BCrypt: A password-hashing function designed by Niels Provos and David Mazières based on the Blowfish cipher, incorporating a salt to protect against rainbow table attacks.

Cgroups (Control Groups): A Linux kernel feature that limits, accounts for, and isolates the resource usage (CPU, memory, disk I/O, network) of a collection of processes.

Hake's Normalized Gain (g): A standard metric in physics and computing education research measuring the fraction of possible cognitive gain achieved by students between pre-test and post-test assessments.

Idempotency: The property of certain operations in mathematics and computer science whereby that operation can be applied multiple times without changing the result beyond the initial application.

Java Virtual Machine (JVM): An abstract computing machine that enables a computer to run a Java program by executing bytecode compiled from Java source code.

JSON Web Token (JWT): An open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object.

Monaco Editor: The code editor engine that powers Visual Studio Code, providing syntax highlighting, IntelliSense autocomplete, bracket matching, and code folding inside web browsers.

Spring Boot: An open-source, microservice-ready Java-based framework used to create stand-alone, production-grade Spring applications with minimal configuration.